

Combining Human and AI Strengths to Find Adversarial Instances for Heuristics

Henri Nikoleit (University of Bonn)

Ankit Anand (Google DeepMind)

Anurag Murty Naredla (University of Manitoba)

Heiko Röglin (University of Bonn)

Bin Packing

0.3

0.5

0.2

0.4

0.2

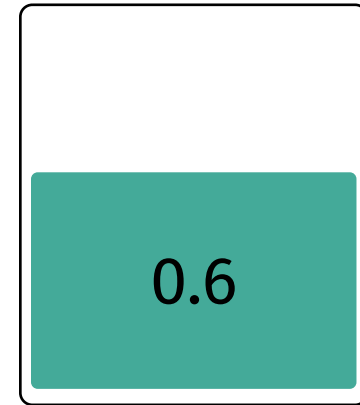
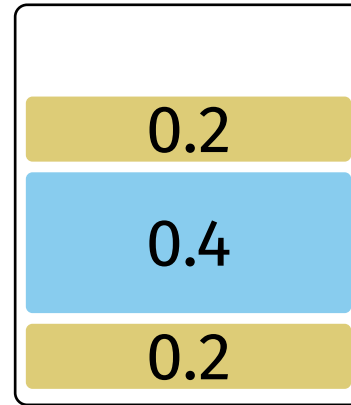
0.6

0.4

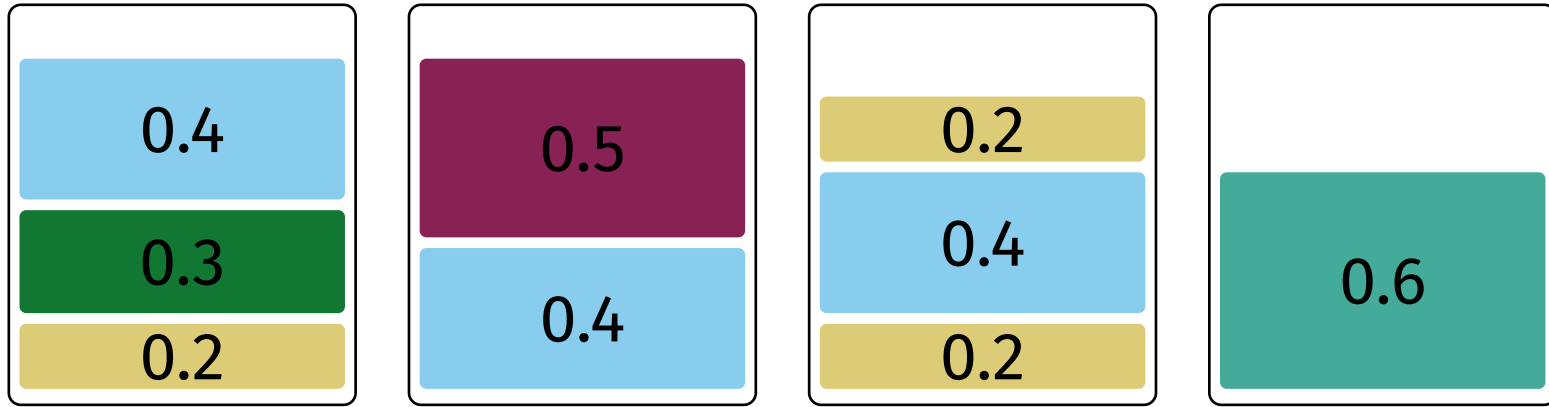
0.2

0.4

Bin Packing



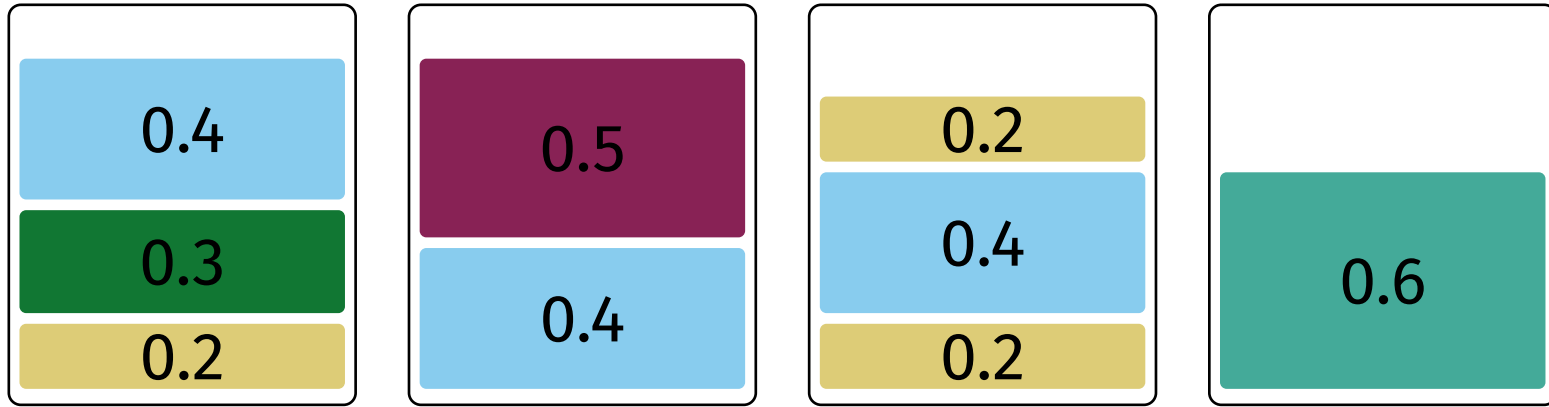
Bin Packing



Task:

- Pack items of sizes $w_1, \dots, w_n \in [0, 1]$ into bins of capacity 1.
-

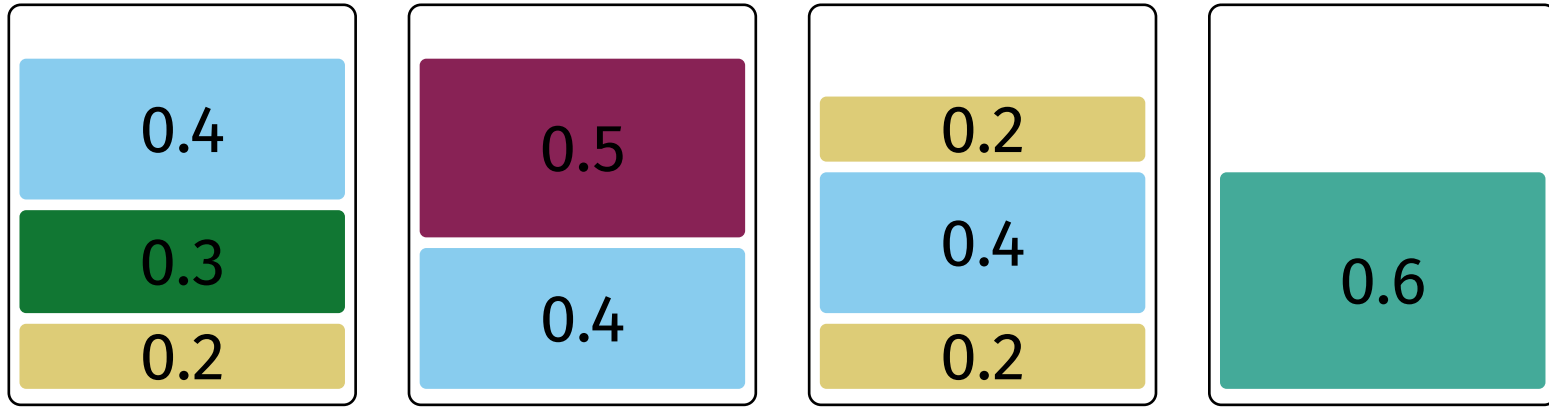
Bin Packing



Task:

- Pack items of sizes $w_1, \dots, w_n \in [0, 1]$ into bins of capacity 1.
- Use as few bins as possible.

Bin Packing



Task:

- Pack items of sizes $w_1, \dots, w_n \in [0, 1]$ into bins of capacity 1.
- Use as few bins as possible.

NP-hard.

Bin Packing: Randomised Best-Fit Heuristic

Consider items in random order. When a new item arrives, put into the bin where it leaves the least remaining space, without overfilling the bin.

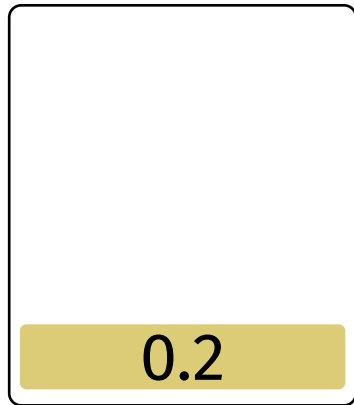
Bin Packing: Randomised Best-Fit Heuristic

Consider items in random order. When a new item arrives, put into the bin where it leaves the least remaining space, without overfilling the bin.

0.2

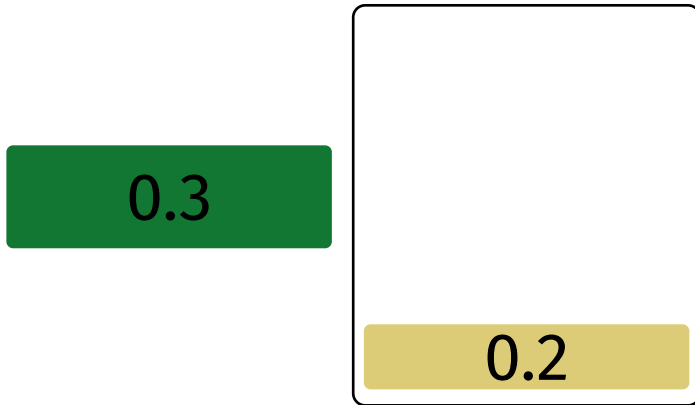
Bin Packing: Randomised Best-Fit Heuristic

Consider items in random order. When a new item arrives, put into the bin where it leaves the least remaining space, without overfilling the bin.



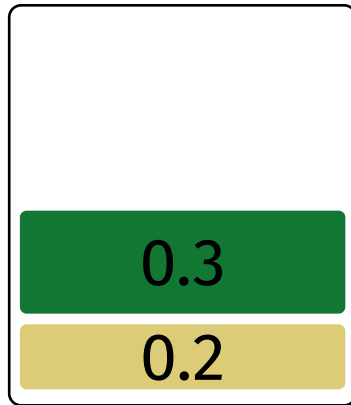
Bin Packing: Randomised Best-Fit Heuristic

Consider items in random order. When a new item arrives, put into the bin where it leaves the least remaining space, without overfilling the bin.



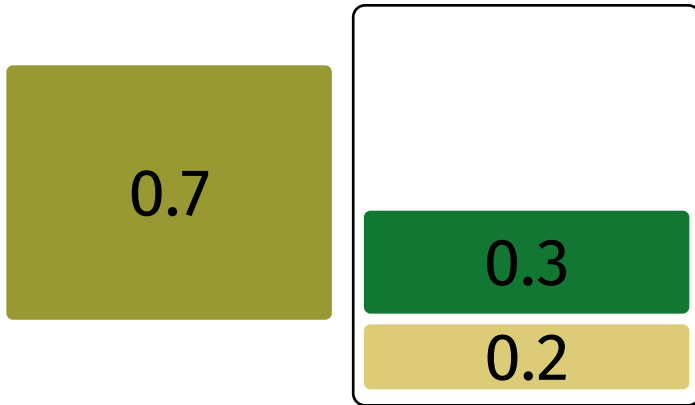
Bin Packing: Randomised Best-Fit Heuristic

Consider items in random order. When a new item arrives, put into the bin where it leaves the least remaining space, without overfilling the bin.



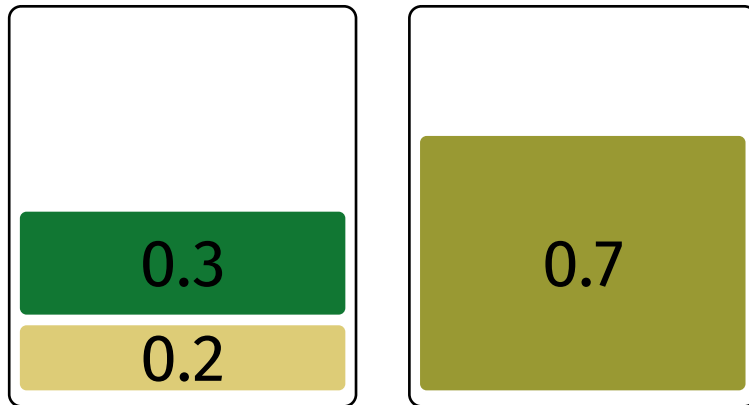
Bin Packing: Randomised Best-Fit Heuristic

Consider items in random order. When a new item arrives, put into the bin where it leaves the least remaining space, without overfilling the bin.



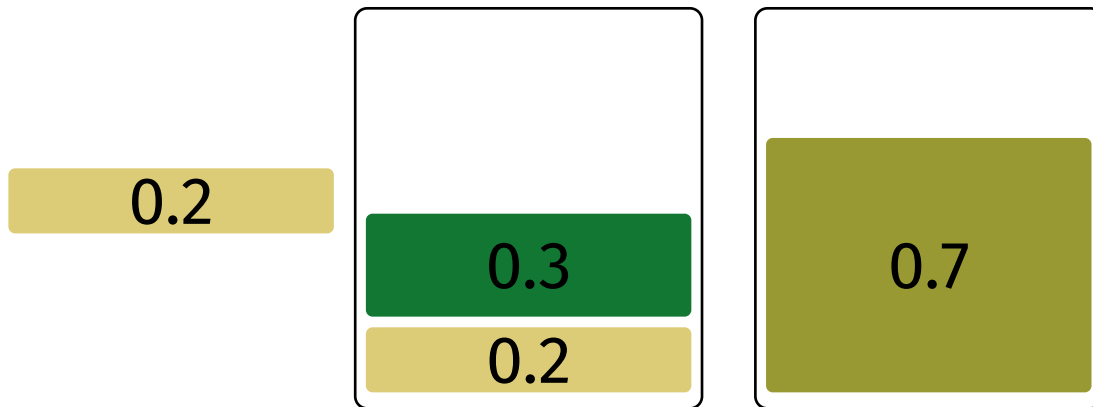
Bin Packing: Randomised Best-Fit Heuristic

Consider items in random order. When a new item arrives, put into the bin where it leaves the least remaining space, without overfilling the bin.



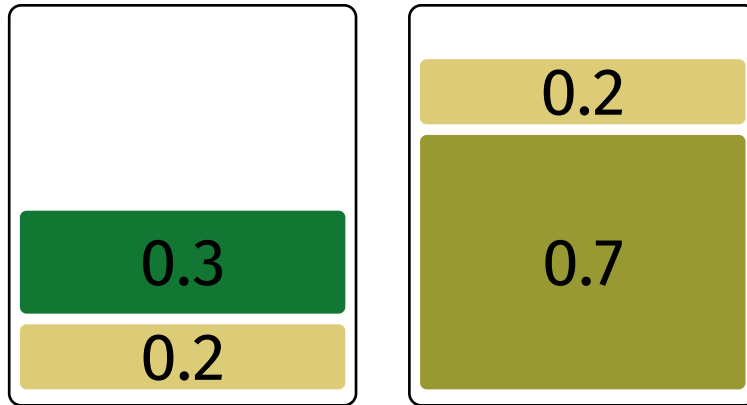
Bin Packing: Randomised Best-Fit Heuristic

Consider items in random order. When a new item arrives, put into the bin where it leaves the least remaining space, without overfilling the bin.



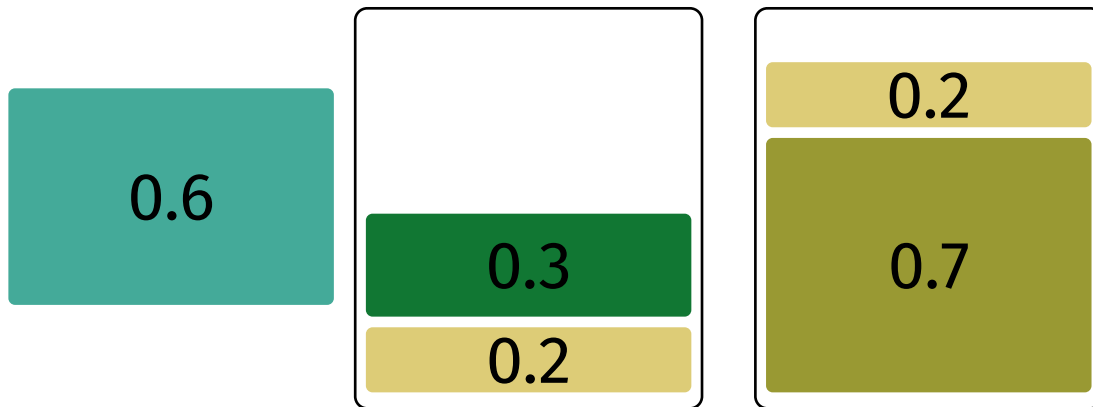
Bin Packing: Randomised Best-Fit Heuristic

Consider items in random order. When a new item arrives, put into the bin where it leaves the least remaining space, without overfilling the bin.



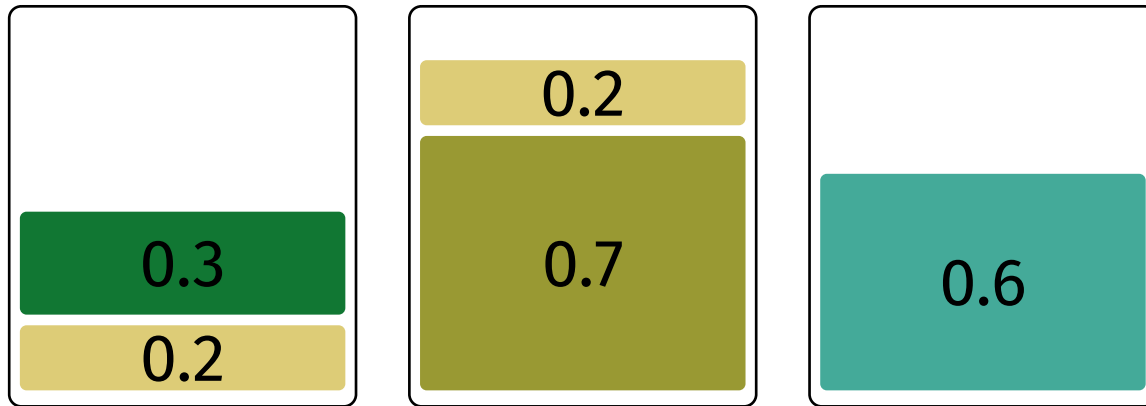
Bin Packing: Randomised Best-Fit Heuristic

Consider items in random order. When a new item arrives, put into the bin where it leaves the least remaining space, without overfilling the bin.



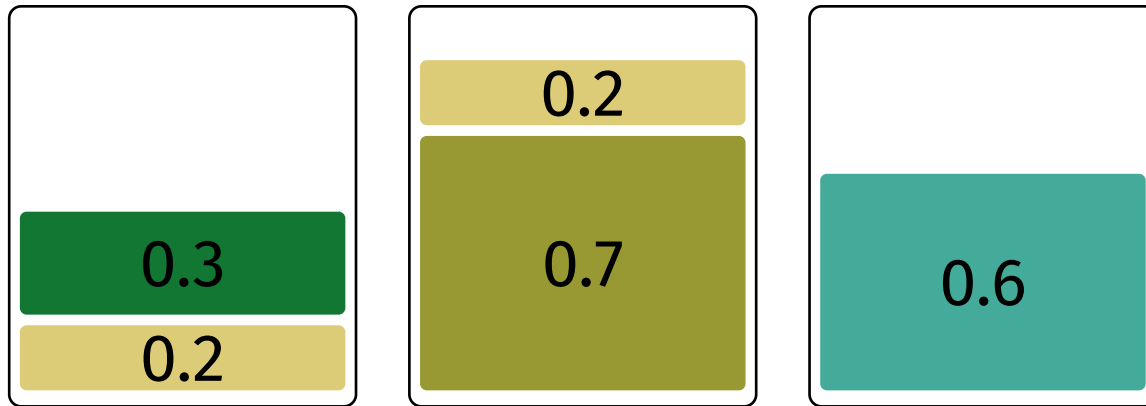
Bin Packing: Randomised Best-Fit Heuristic

Consider items in random order. When a new item arrives, put into the bin where it leaves the least remaining space, without overfilling the bin.



Bin Packing: Randomised Best-Fit Heuristic

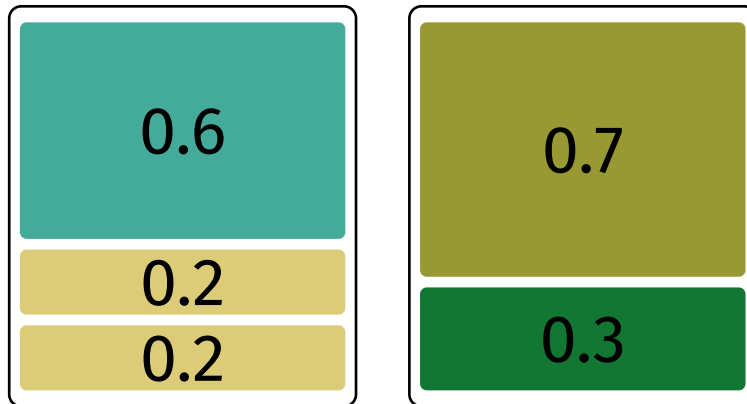
Consider items in random order. When a new item arrives, put into the bin where it leaves the least remaining space, without overfilling the bin.



Bin Packing: Randomised Best-Fit Heuristic

Consider items in random order. When a new item arrives, put into the bin where it leaves the least remaining space, without overfilling the bin.

Optimal:



How well does Randomised Best-Fit perform compared to the optimum?

- “ $\text{Opt}(I)$ ” is the number of bins in an optimal solution on instance I
-

How well does Randomised Best-Fit perform compared to the optimum?

- “ $\text{Opt}(I)$ ” is the number of bins in an optimal solution on instance I
- $\text{RandBestFit}(I)$ the *expected* number of bins used by Randomised Best Fit

How well does Randomised Best-Fit perform compared to the optimum?

- “ $\text{Opt}(I)$ ” is the number of bins in an optimal solution on instance I
- $\text{RandBestFit}(I)$ the *expected* number of bins used by Randomised Best Fit

How large can $\text{RandBestFit}(I) / \text{Opt}(I)$ get?

How well does Randomised Best-Fit perform compared to the optimum?

- “ $\text{Opt}(I)$ ” is the number of bins in an optimal solution on instance I
- $\text{RandBestFit}(I)$ the *expected* number of bins used by Randomised Best Fit

How large can $\text{RandBestFit}(I) / \text{Opt}(I)$ get?

$$\text{Apx}_{\text{RandBestFit}} := \sup_I \frac{\text{RandBestFit}(I)}{\text{Opt}(I)}$$

Previous Work on $\text{Ap}^{\text{X}}_{\text{RandBestFit}}$

Previous Work on $\text{Apx}_{\text{RandBestFit}}$

$$1.3 \quad (\text{Albers et al., 2021}) \quad \leq \quad \text{Apx}_{\text{RandBestFit}} \quad (\text{Dósa \& Sgall, 2014}) \quad \leq \quad 1.7.$$

Previous Work on $\text{Apx}_{\text{RandBestFit}}$

$$1.3 \stackrel{(\text{Albers et al., 2021})}{\leq} \text{Apx}_{\text{RandBestFit}} \stackrel{(\text{Dósa \& Sgall, 2014})}{\leq} 1.7.$$

How could **you** try to improve the lower bound?

Local Search

Let $\text{Score}(I) := \text{RandBestFit}(I) / \text{Opt}(I)$.

1.

Local Search

Let $\text{Score}(I) := \text{RandBestFit}(I) / \text{Opt}(I)$.

1. Start with some bin-packing instance I (list of floats).
- 2.

Local Search

Let $\text{Score}(I) := \text{RandBestFit}(I) / \text{Opt}(I)$.

1. Start with some bin-packing instance I (list of floats).
2. Throw some random noise on I :

$$I' = [x + \text{random.random}() \mid x \in I].$$

- 3.

Local Search

Let $\text{Score}(I) := \text{RandBestFit}(I) / \text{Opt}(I)$.

1. Start with some bin-packing instance I (list of floats).
2. Throw some random noise on I :

$$I' = [x + \text{random.random}() \mid x \in I].$$

3. Check if $\text{Score}(I') > \text{Score}(I)$.

•

Local Search

Let $\text{Score}(I) := \text{RandBestFit}(I) / \text{Opt}(I)$.

1. Start with some bin-packing instance I (list of floats).
2. Throw some random noise on I :

$$I' = [x + \text{random.random}() \mid x \in I].$$

3. Check if $\text{Score}(I') > \text{Score}(I)$.
 - If so, replace I with I'
- 4.

Local Search

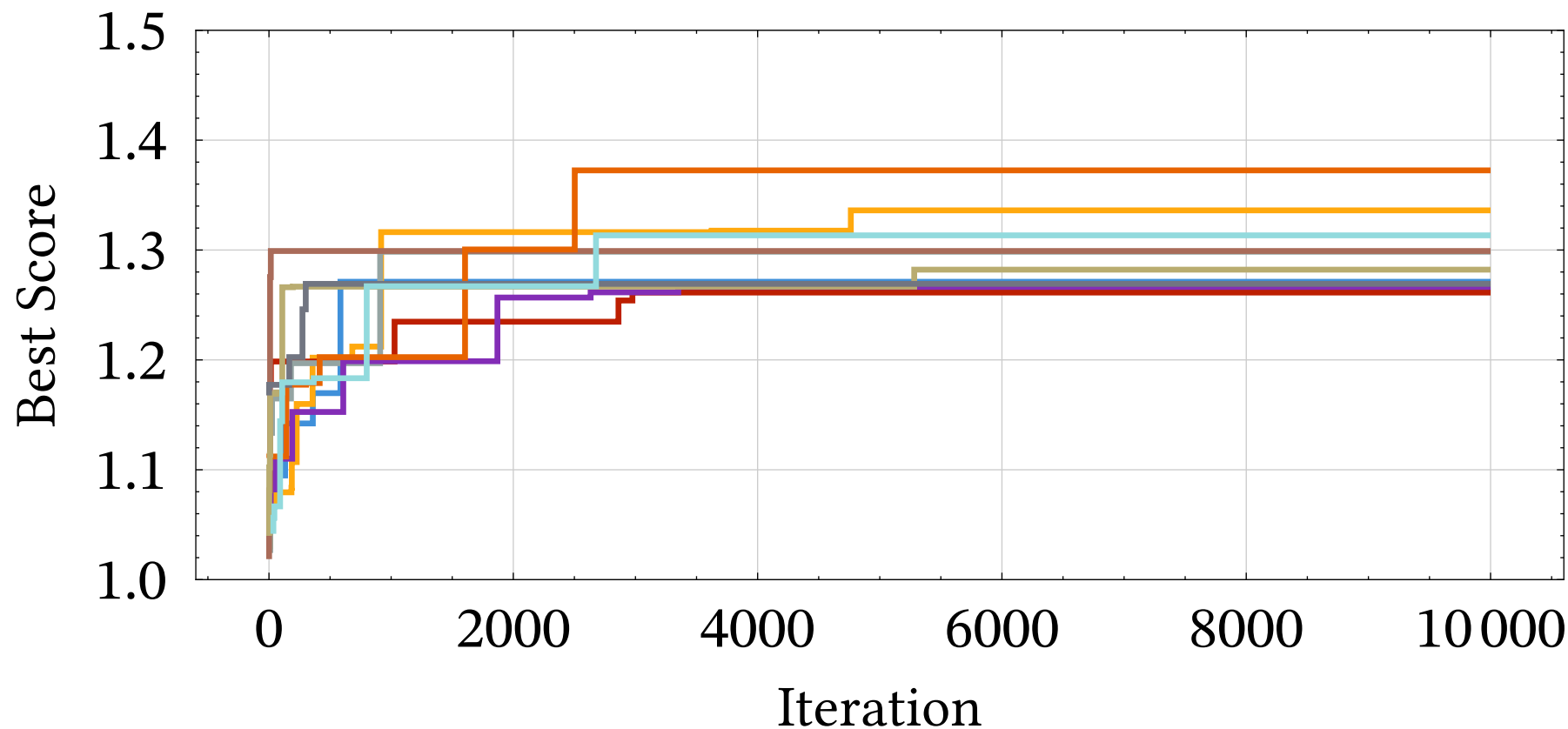
Let $\text{Score}(I) := \text{RandBestFit}(I) / \text{Opt}(I)$.

1. Start with some bin-packing instance I (list of floats).
2. Throw some random noise on I :

$$I' = [x + \text{random.random}() \mid x \in I].$$

3. Check if $\text{Score}(I') > \text{Score}(I)$.
 - If so, replace I with I'
4. Goto 2.

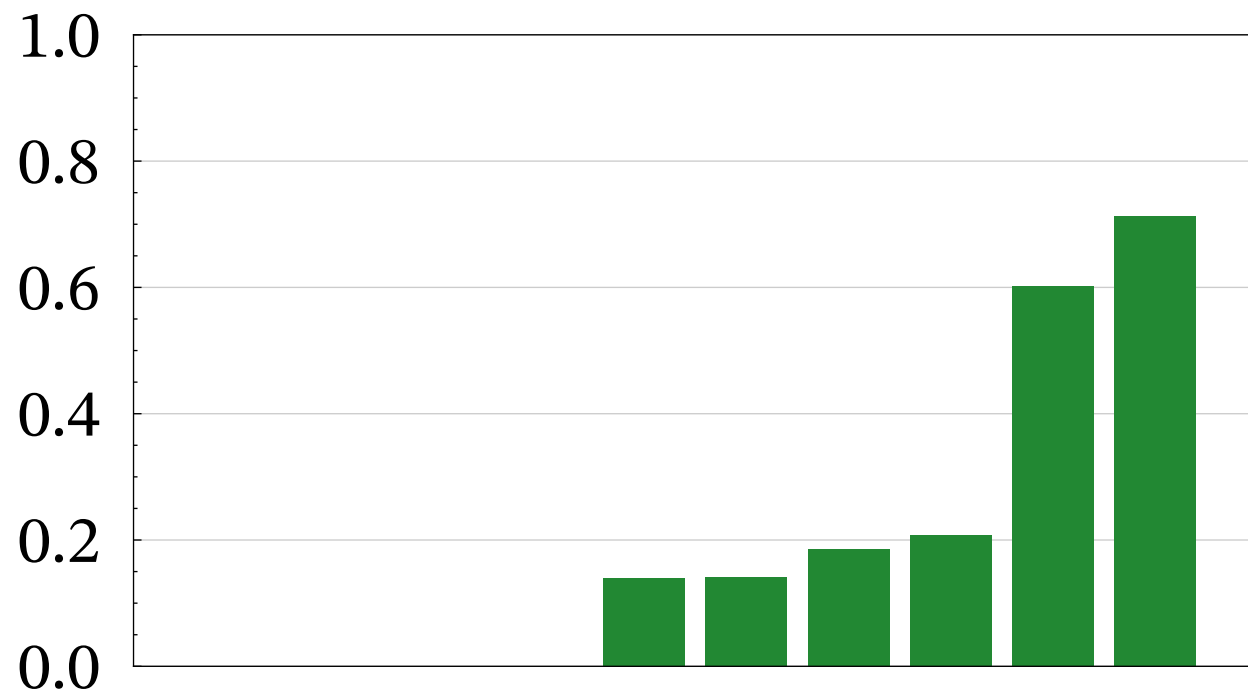
Ten Local Search Trajectories

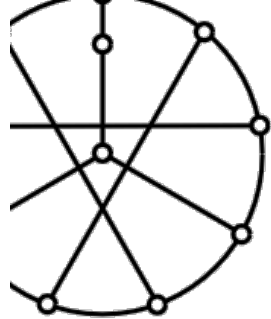


Examining the best instance

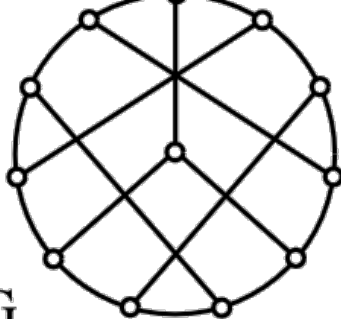
Examining the best instance

```
[0, 0, 0, 0,  
0.13941968656458636,  
0.1415175313246237,  
0.18488603733618258,  
0.20818251654978343,  
0.6014145332633378,  
0.7129758245684663]
```

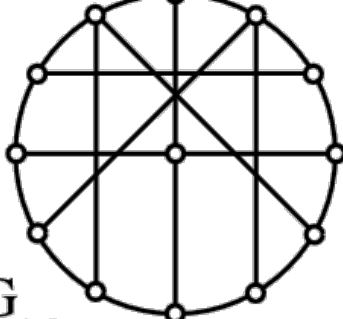




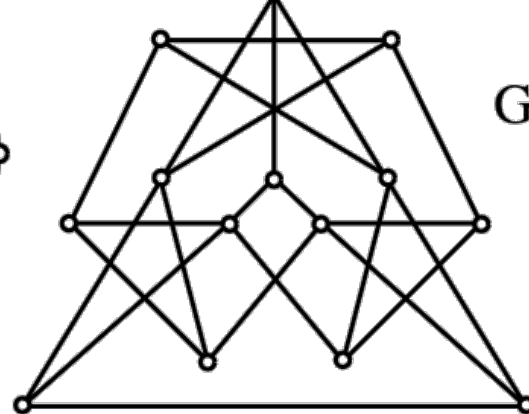
11



G_{12}

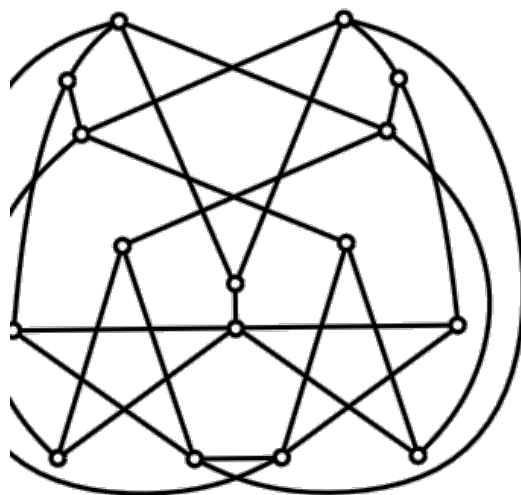
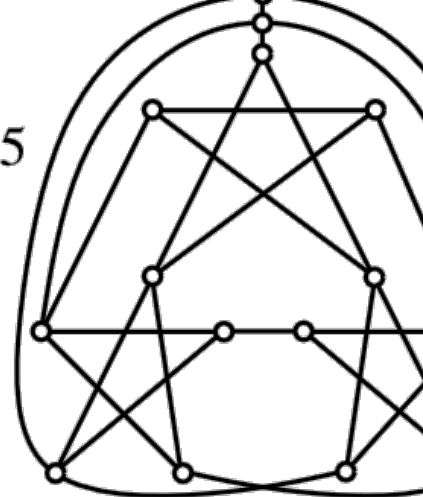


G_{13}

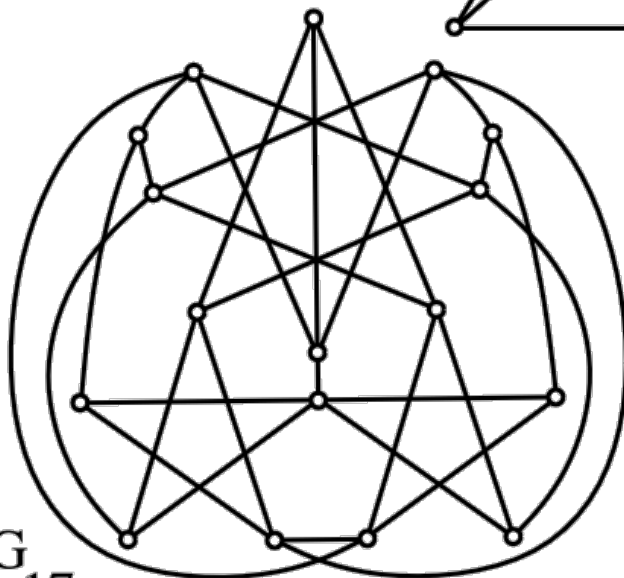


G_{14}

G_{15}

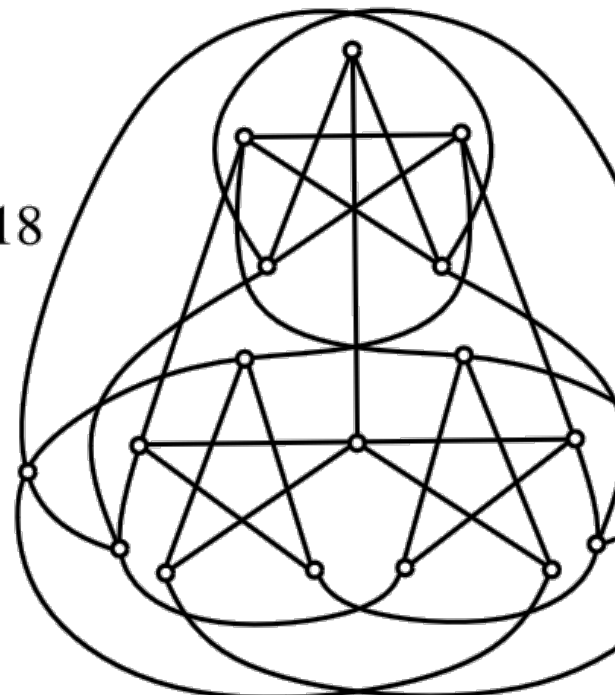


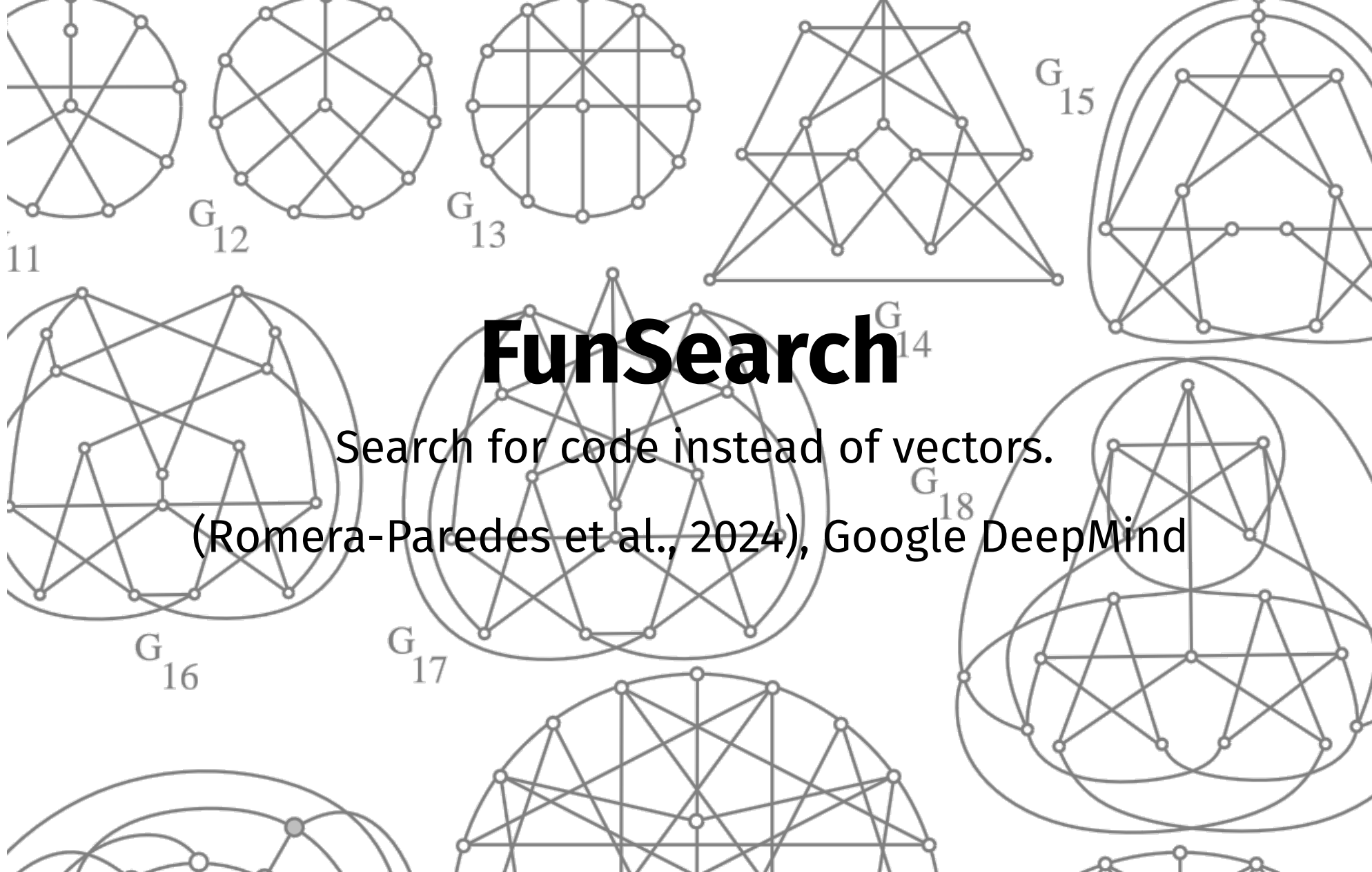
G_{16}



G_{17}

G_{18}





FunSearch

Search for code instead of vectors.

(Romera-Paredes et al., 2024), Google DeepMind

Local Search

Let $\text{Score}(I) := \text{RandBestFit}(I) / \text{Opt}(I)$.

1. Start with some list of floats I .
2. Throw some random noise on I :

$$I' = [x + \text{random.random}() \mid x \in I].$$

3. Check if $\text{Score}(I') > \text{Score}(I)$.
 - If so, replace I with I'
4. Goto 2.

~~Local Search~~ FunSearch

Let $\text{Score}(I) := \text{RandBestFit}(I) / \text{Opt}(I)$.

1. Start with some ~~list of floats~~ `python-program` I .
2. Throw some random noise on I :

$$I' = [x + \text{random.random}() \mid x \in I].$$

$I' = \text{llm-query}(\text{"Hello ChatGPT, please slightly change } I\ldots\text{"})$.

3. Check if $\text{Score}(I') > \text{Score}(I)$.
 - If so, replace I with I'
4. Goto 2.

Example starting code

```
@funsearch.evolve
def get_items() → list[float]:
    """Return a new bin-packing-instance, specified by the list
    of items. The items must be floats between 0 and 1."""
    items = [0.4, 0.5, 0.6]
    return items
```

Score: 1.0

Function found by FunSearch

```
def get_items() → list[float]:  
    """Yet another version of `get_items_v0`, `get_items_v1`, and  
    `get_items_v2`, with some lines altered."""  
    items = [0.8, 0.2, 0.6, 0.4]  
    # Split the first item into seven smaller items and the fourth  
    item into five smaller items  
    items = [0.114, 0.114, 0.114, 0.114, 0.114, 0.114, 0.114] +  
    items[1:3] + [0.08, 0.08, 0.08, 0.08, 0.08]  
    return items
```

Score: 1.4975

Cleaned up by hand

```
def get_items() → list[float]:  
    return [0.8 / 7] * 7 + [0.2, 0.6] + [0.4 / 5] * 5
```

Score: 1.49

Cleaned up some more

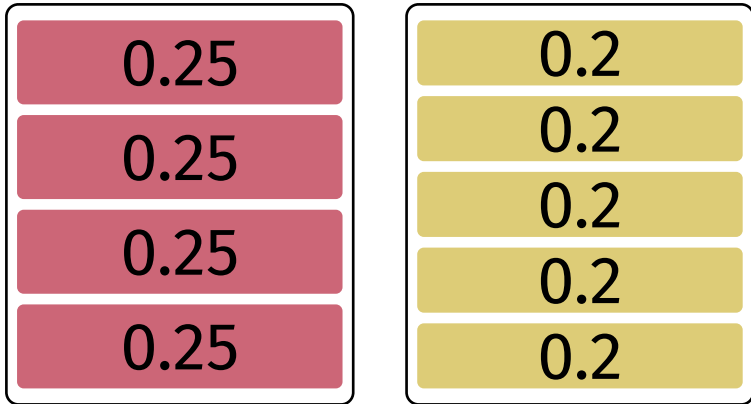
```
def get_items() → list[float]:  
    return [1.0 / 4] * 4 + [1.0 / 5] * 5
```

Score: 1.49

Cleaned up some more

```
def get_items() → list[float]:  
    return [1.0 / 4] * 4 + [1.0 / 5] * 5
```

Score: 1.49



Cleaned up some more

```
def get_items() → list[float]:  
    return [1.0 / 4] * 4 + [1.0 / 5] * 5
```

Score: 1.49



Theorem: The absolute random-order-ratio of Best-Fit $\text{ApX}_{\text{RandBestFit}}$ is at least 1.5.

Best-Fit Absolute

Gasoline 1d

Hierarchical k-median

Hierarchical k-median L_2

Greedy k-means

Greedy k-median L_2^2

Page-Replacement

Best-Fit Asymptotic

Gasoline 2d

Hierarchical k-means

Hierarchical k-median L_2^2

Greedy k-median

Nemhauser-Ullmann

Best-Fit Absolute

Gasoline 1d

Hierarchical k-median

Hierarchical k-median L_2

Greedy k-means

Greedy k-median L_2^2

Page-Replacement

Best-Fit Asymptotic

Gasoline 2d

Hierarchical k-means

Hierarchical k-median L_2^2

Greedy k-median

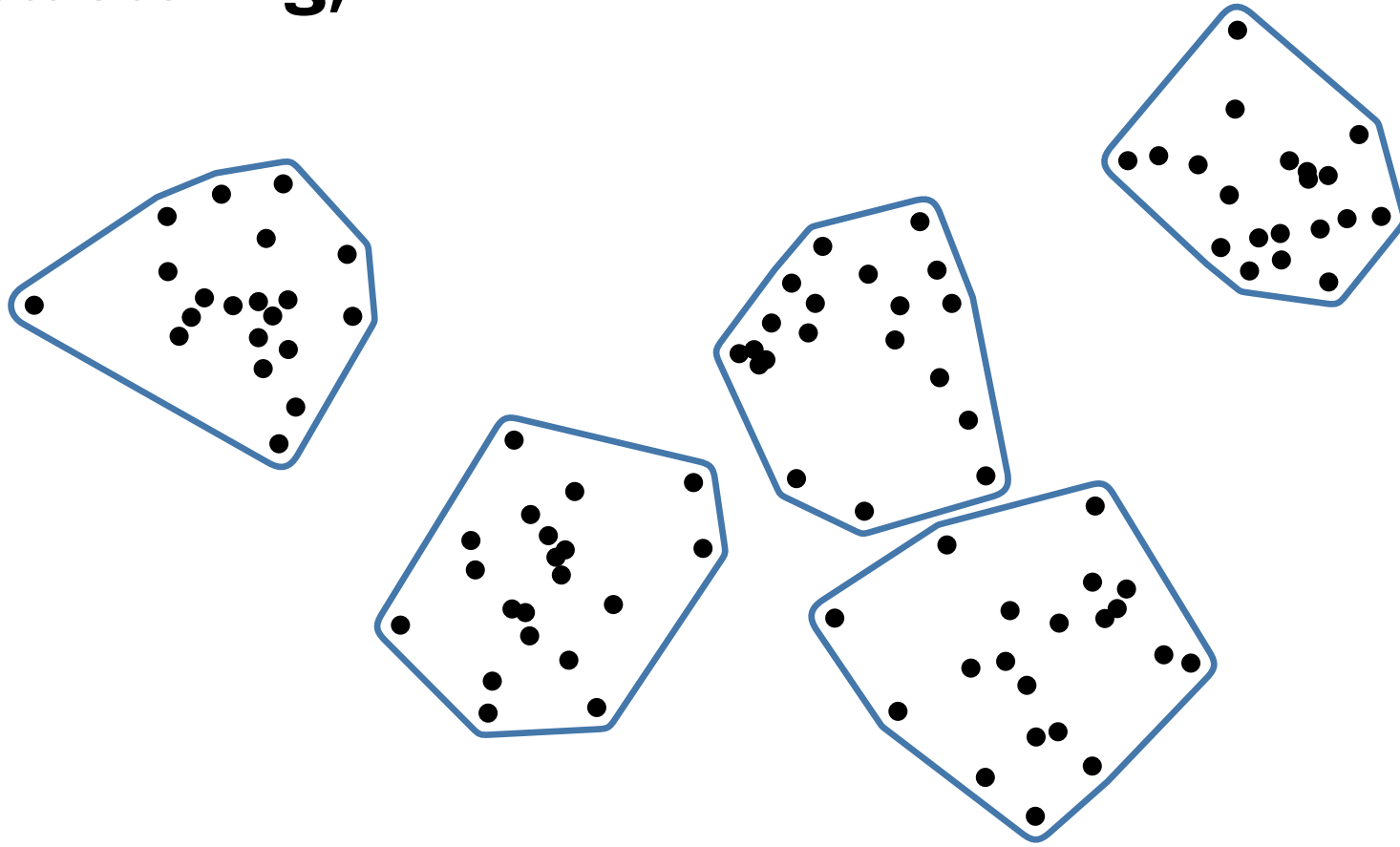
Nemhauser-Ullmann

Clustering

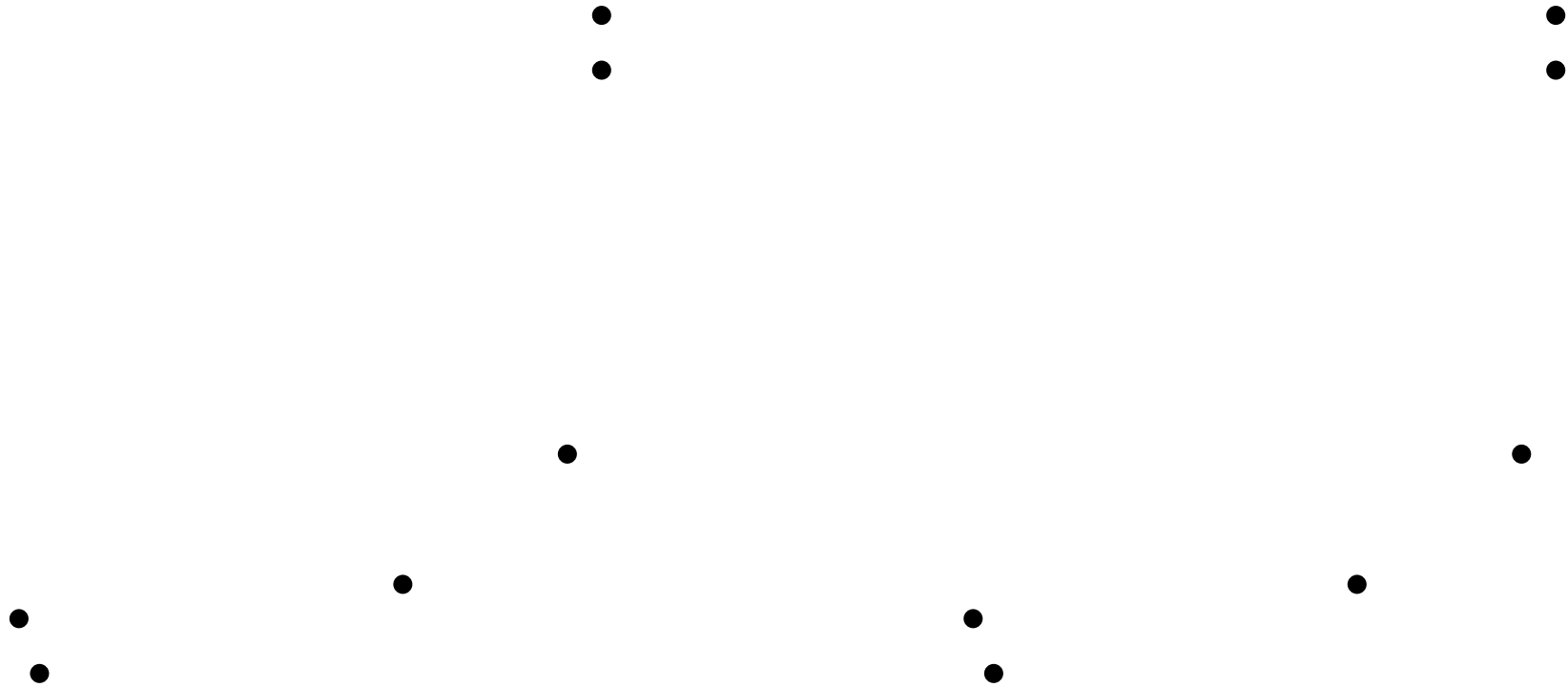
Clustering, $k = 5$



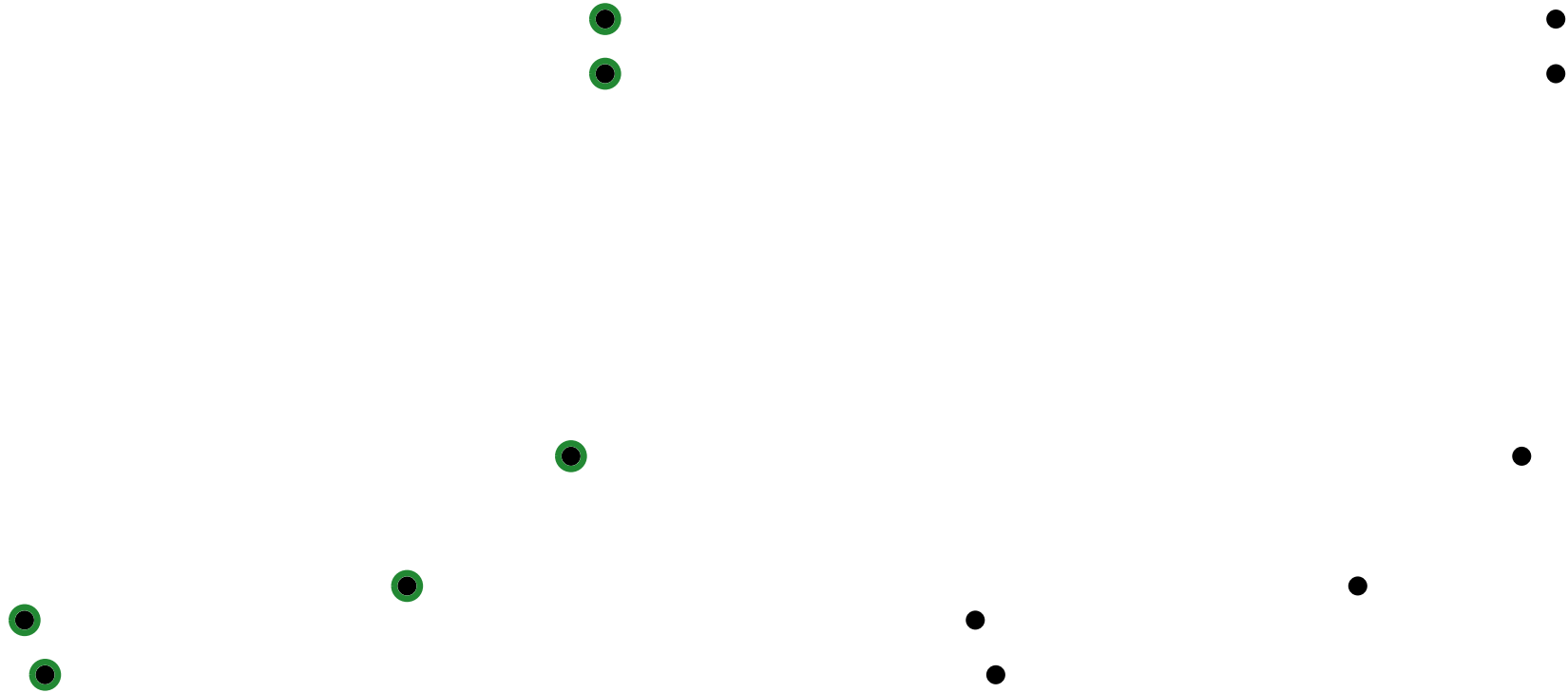
Clustering, $k = 5$



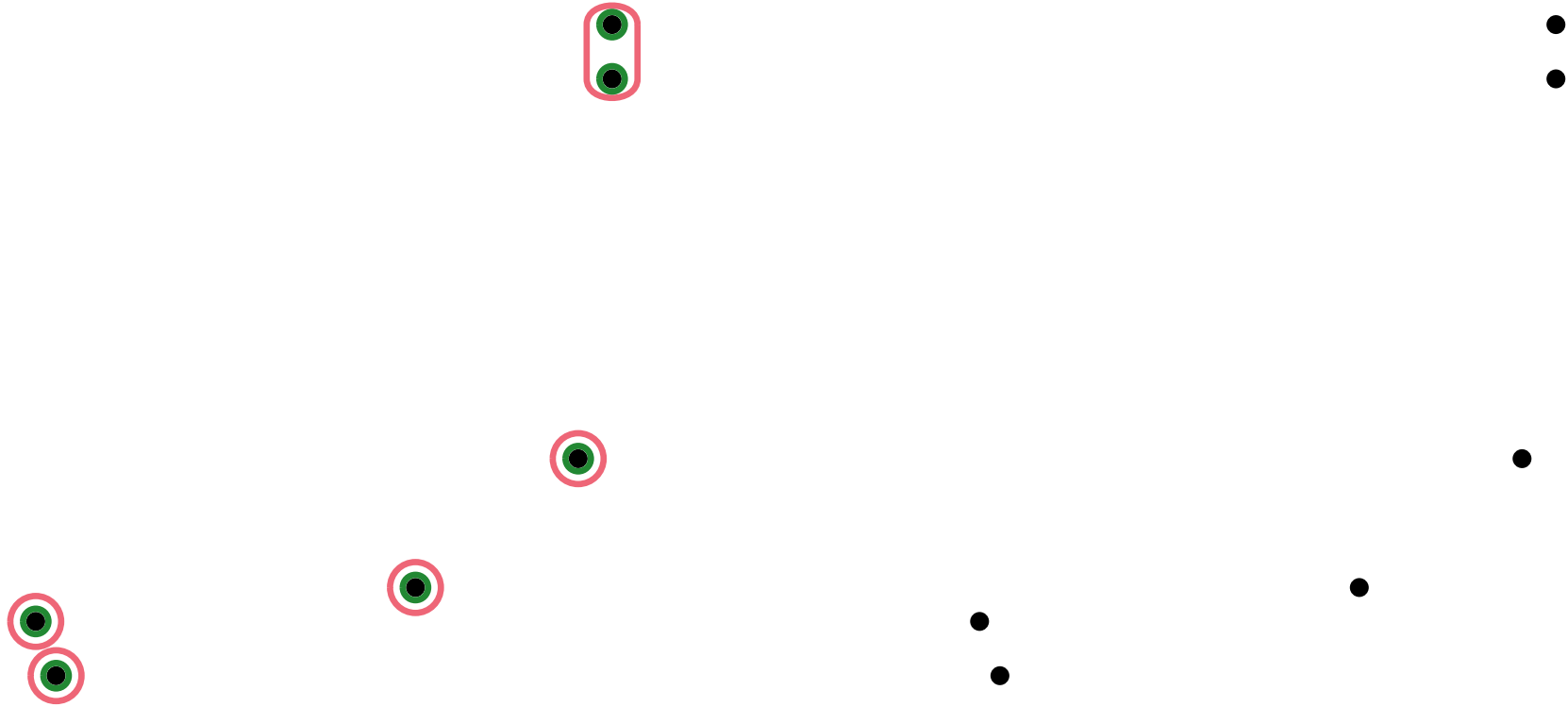
Hierarchical Clustering



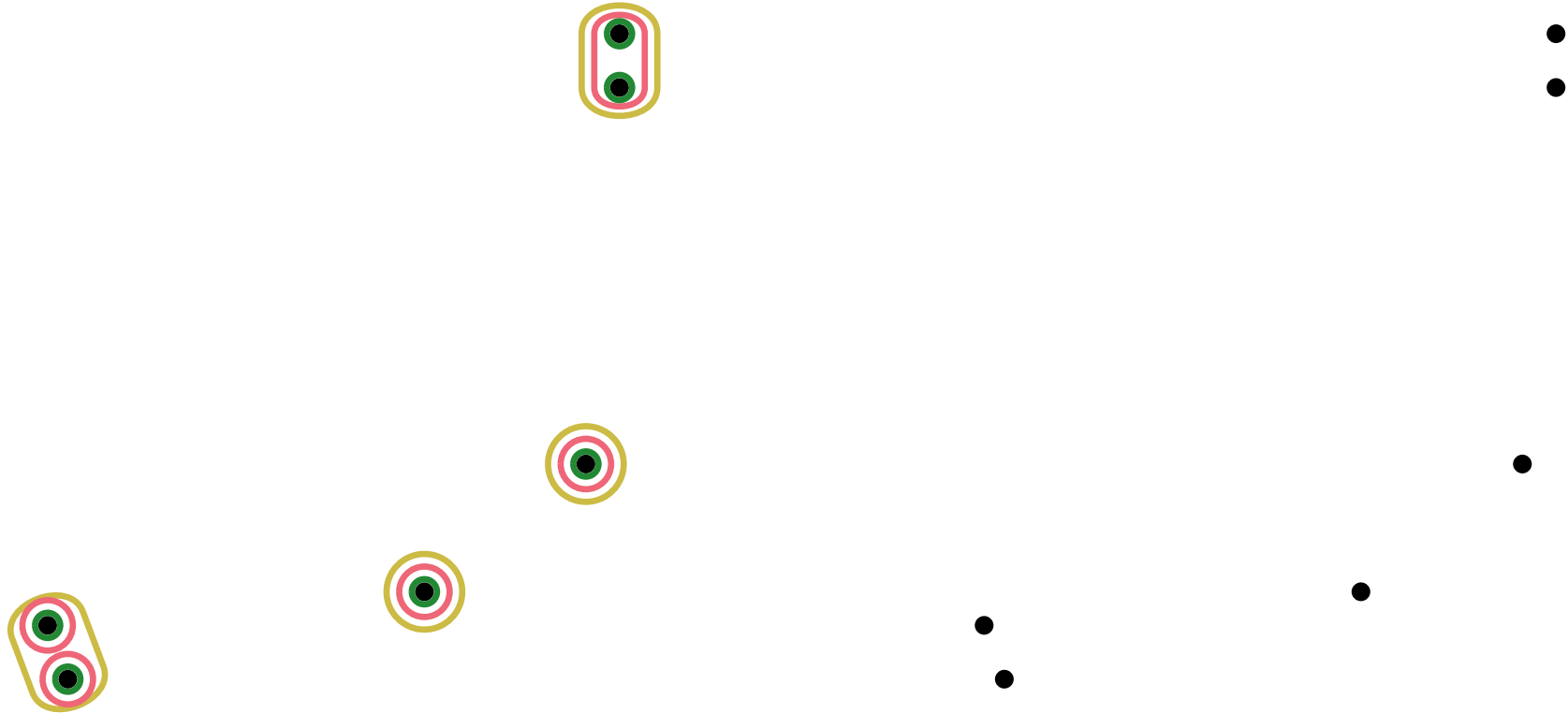
Hierarchical Clustering



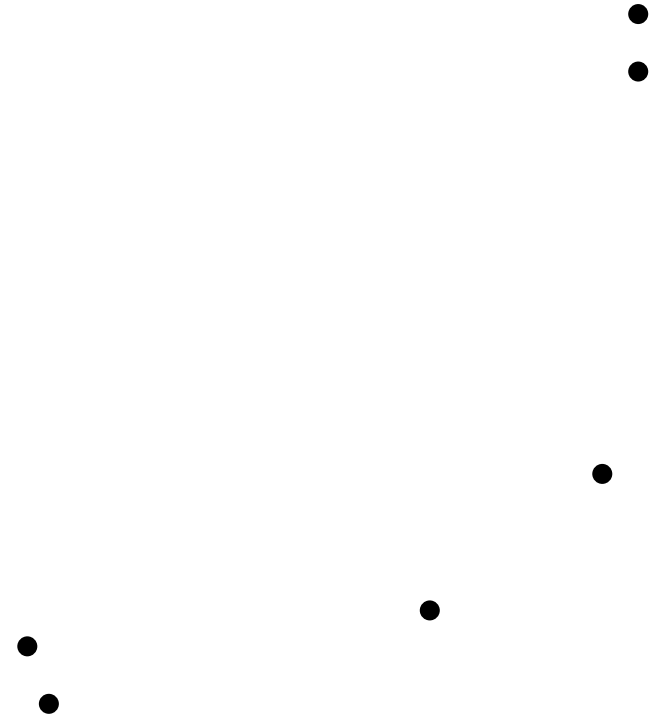
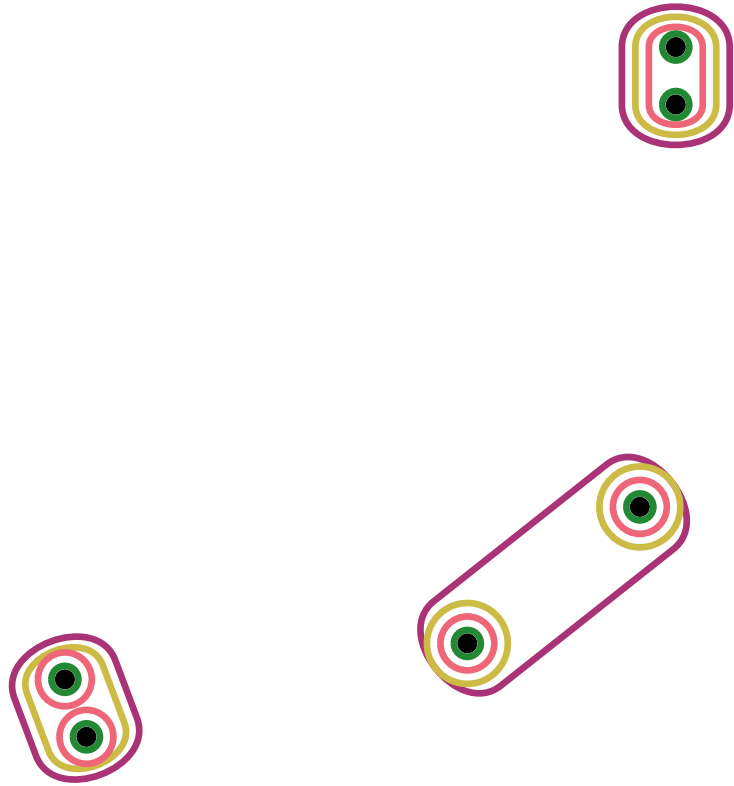
Hierarchical Clustering



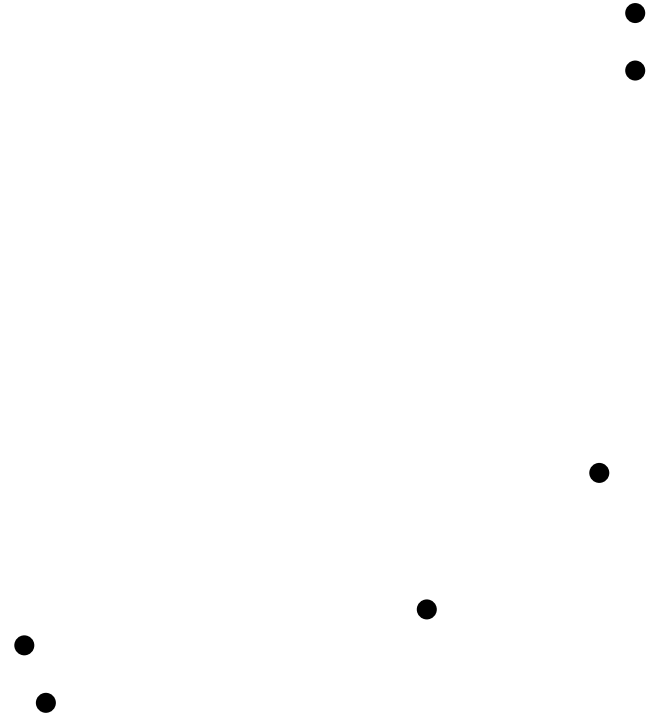
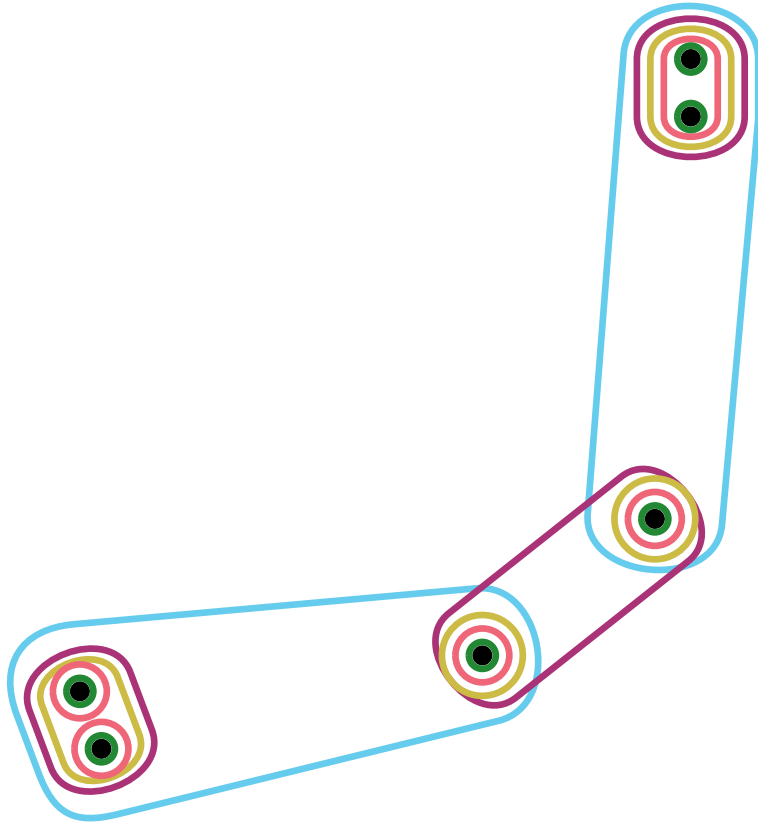
Hierarchical Clustering



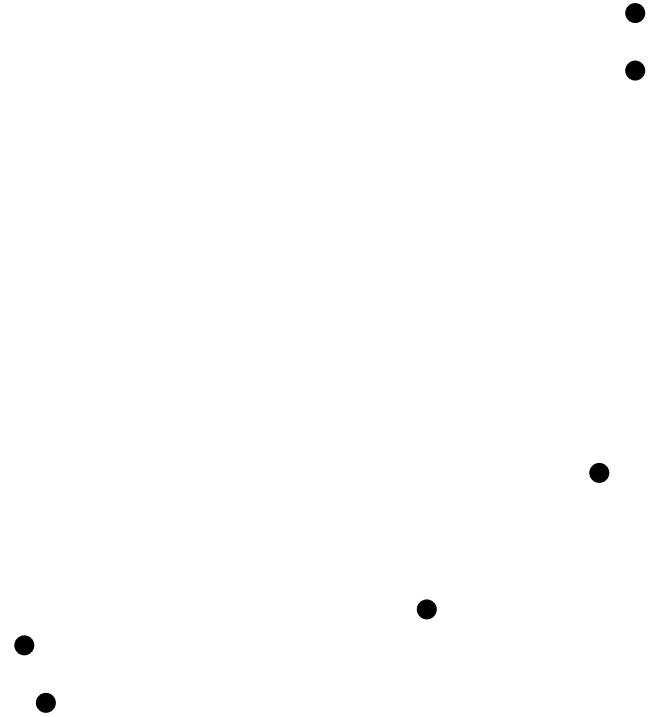
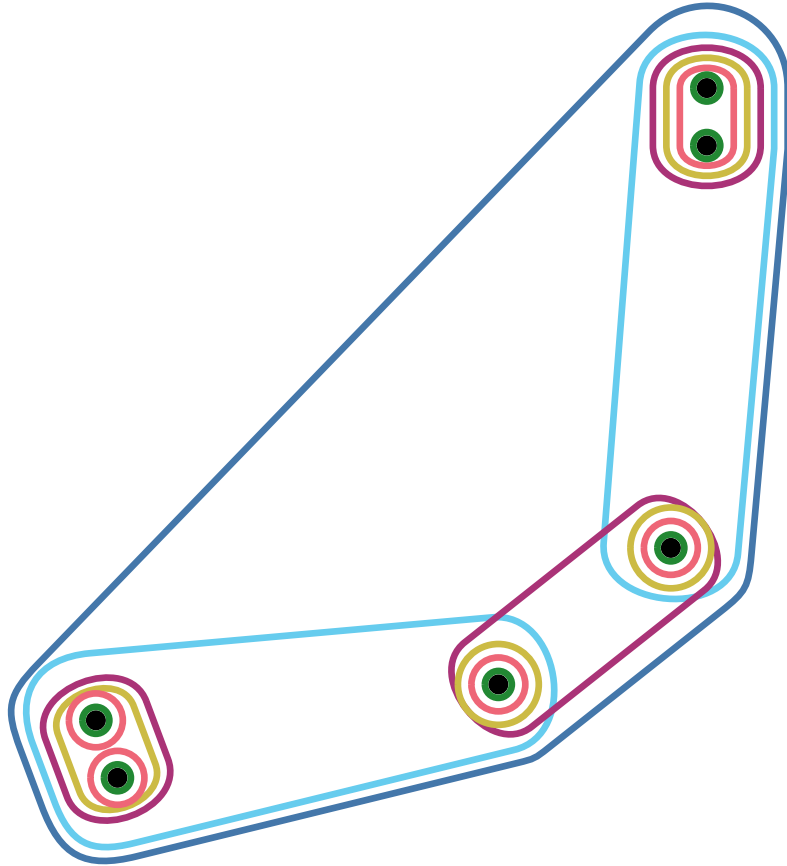
Hierarchical Clustering



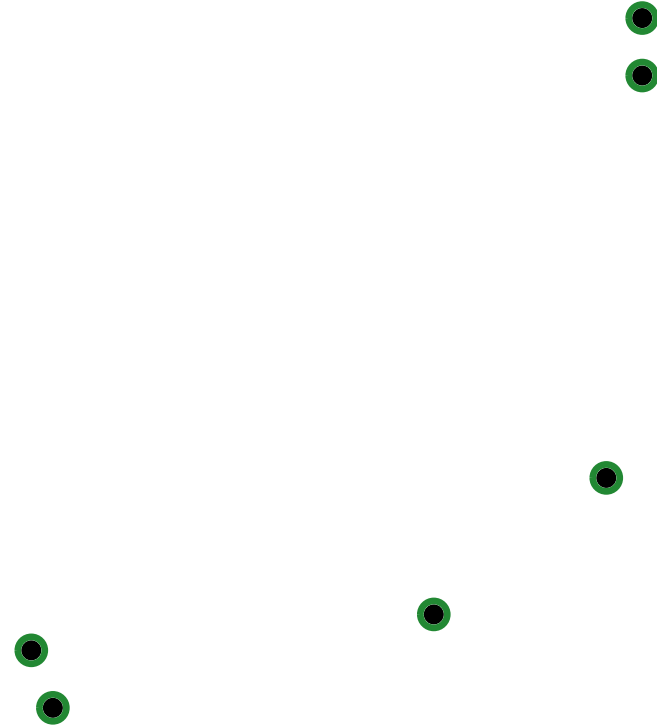
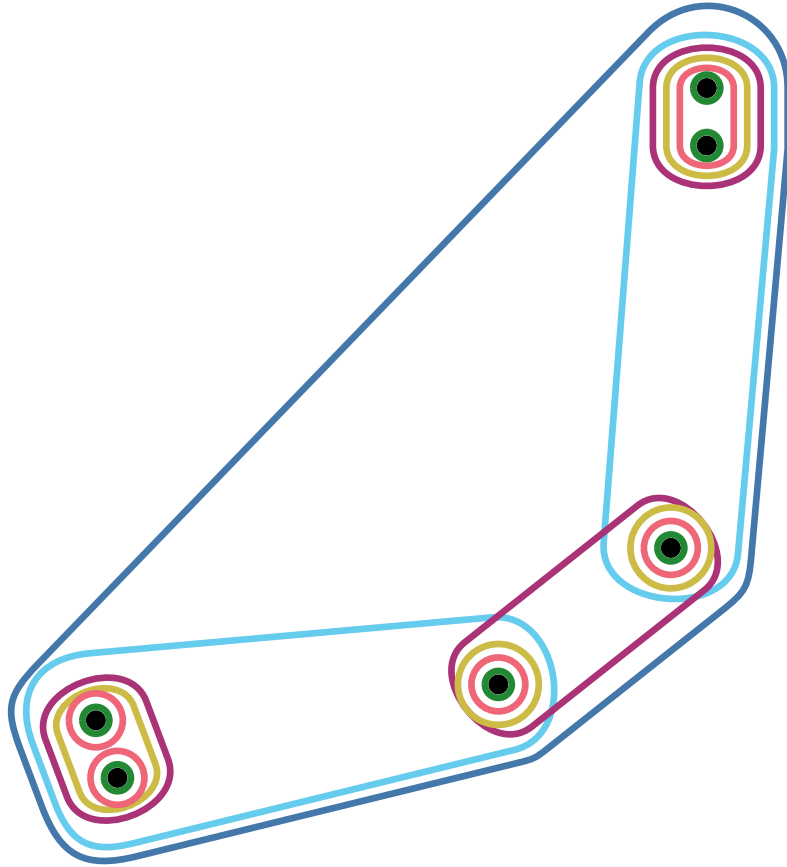
Hierarchical Clustering



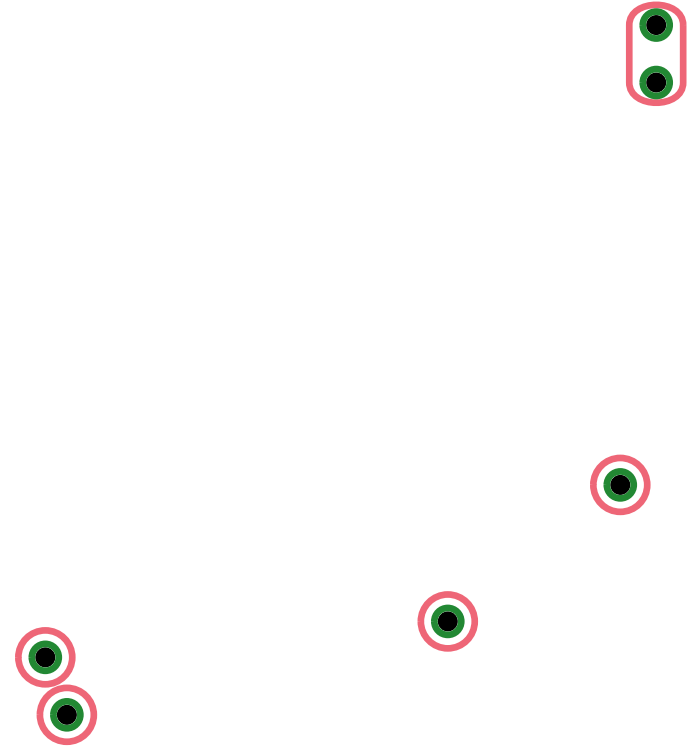
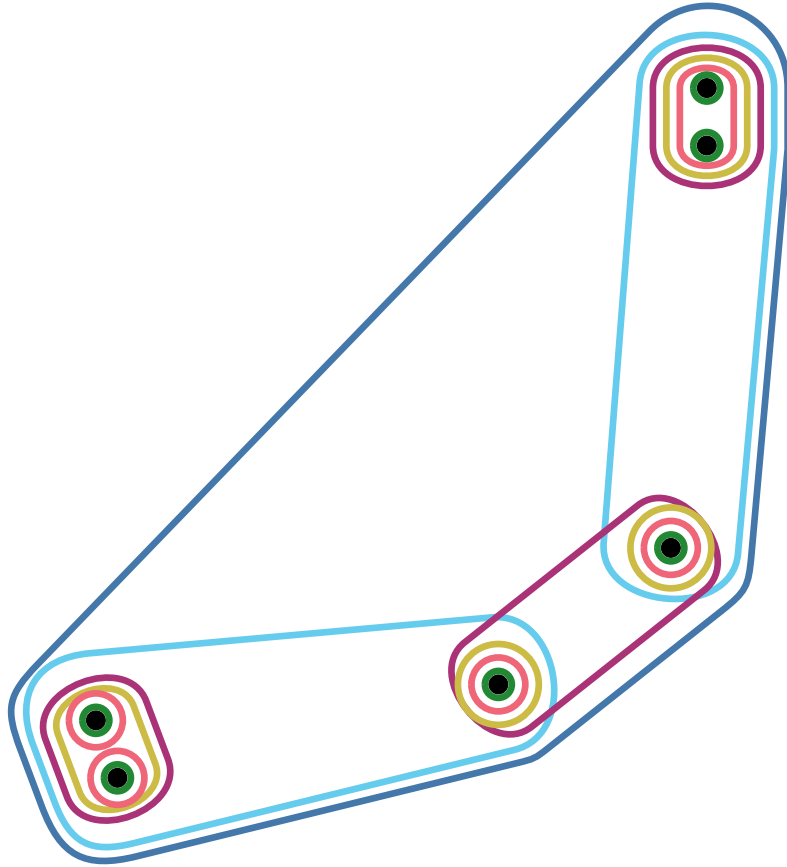
Hierarchical Clustering



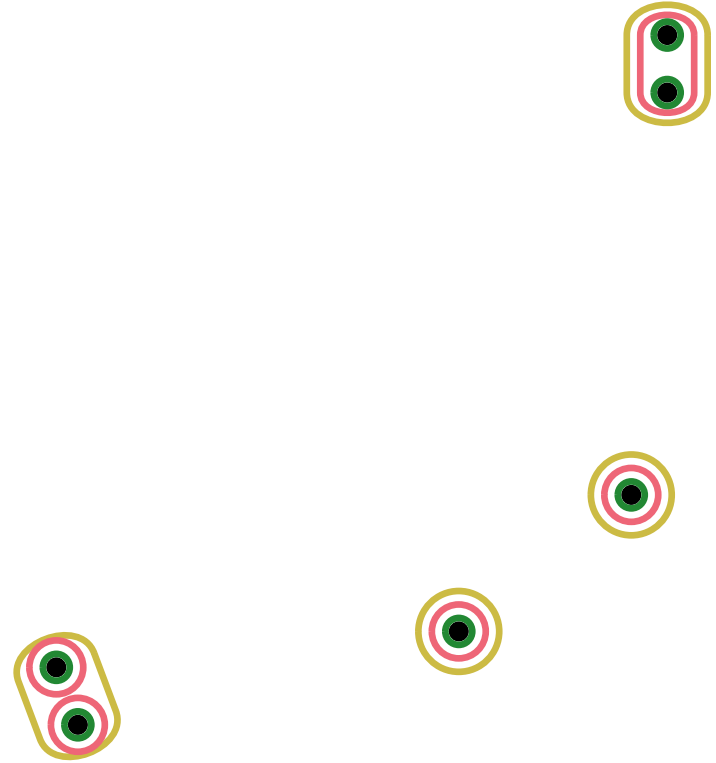
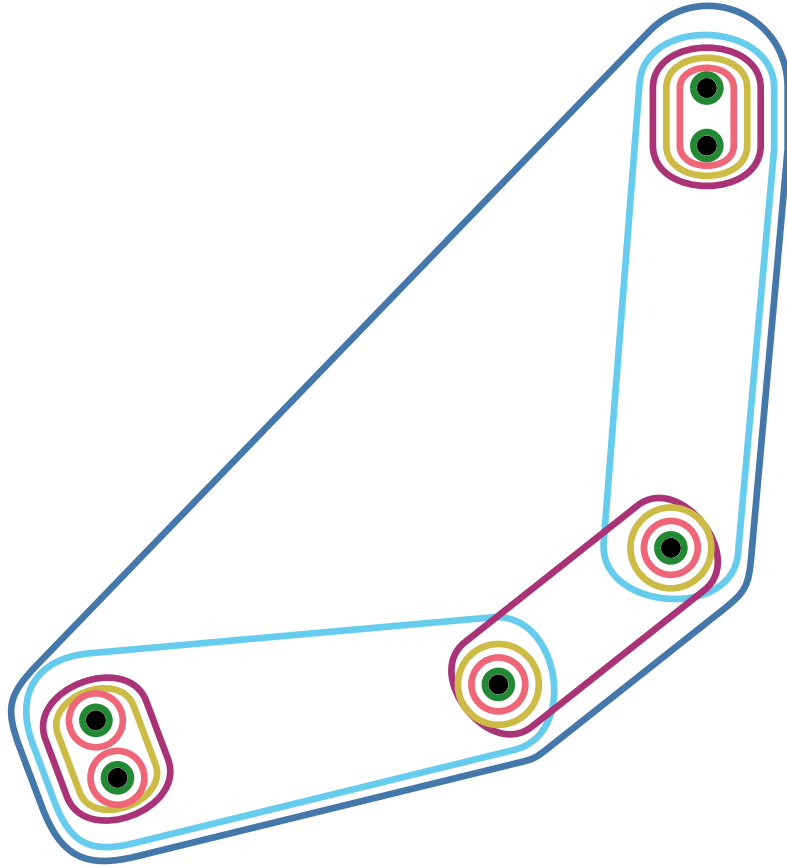
Hierarchical Clustering



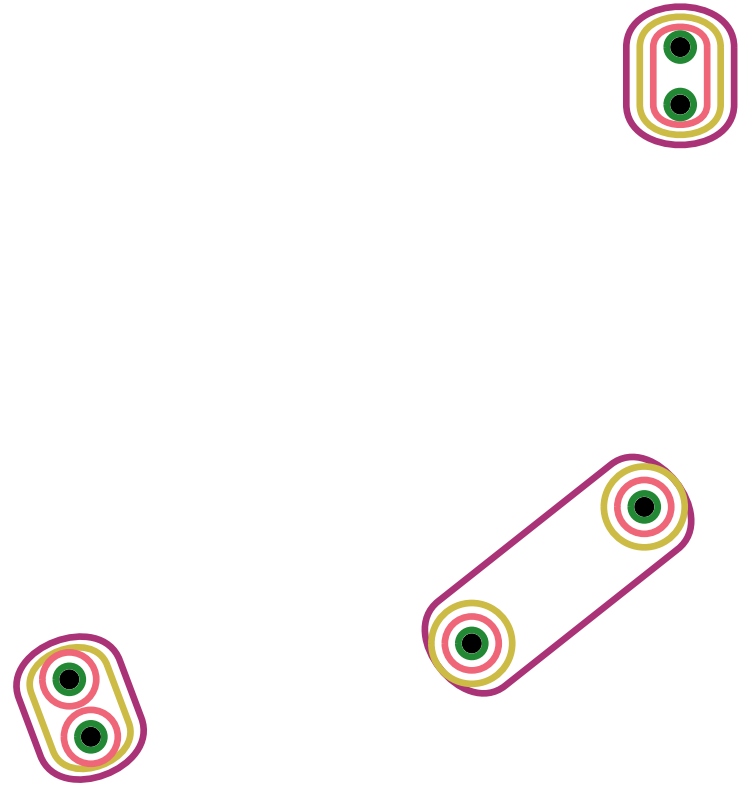
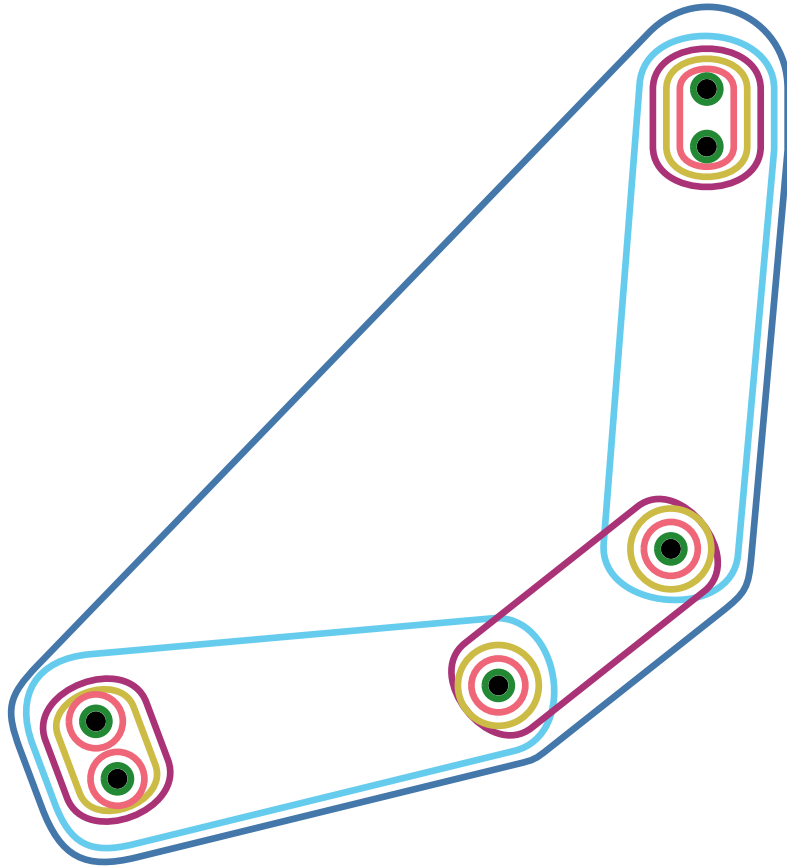
Hierarchical Clustering



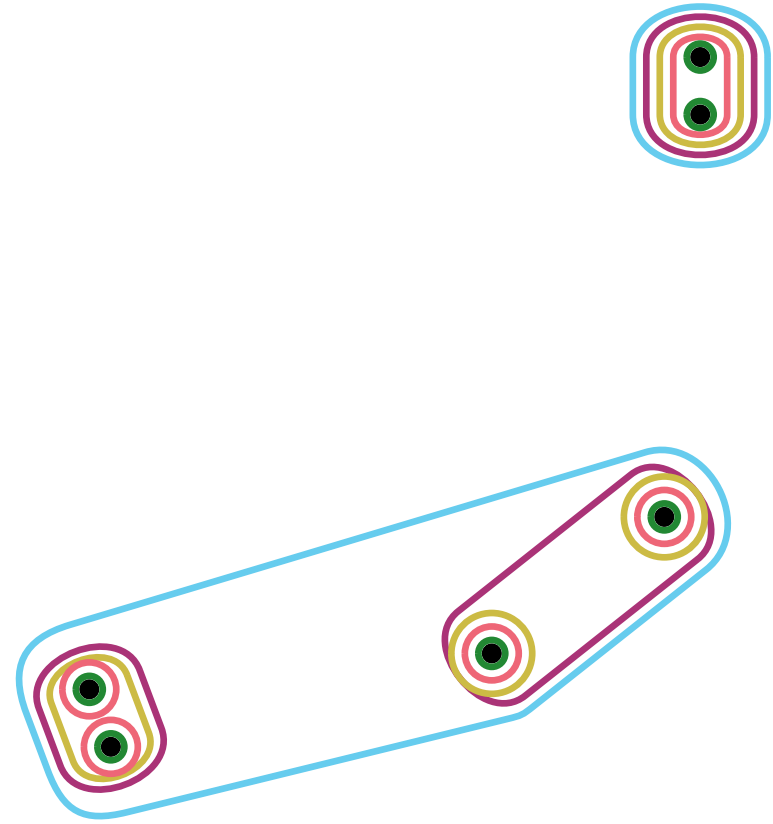
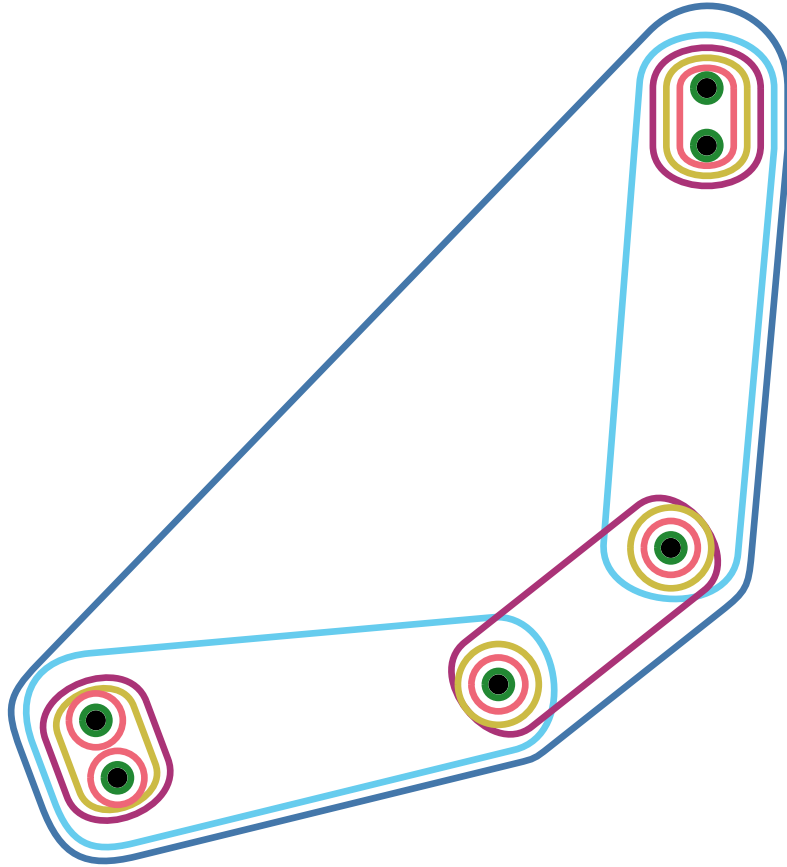
Hierarchical Clustering



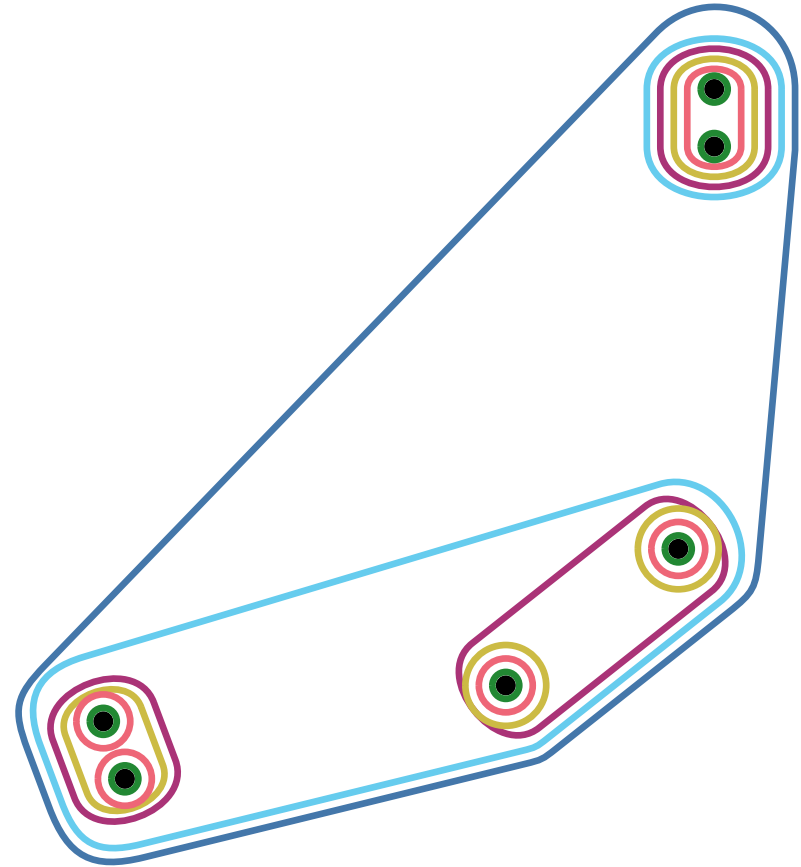
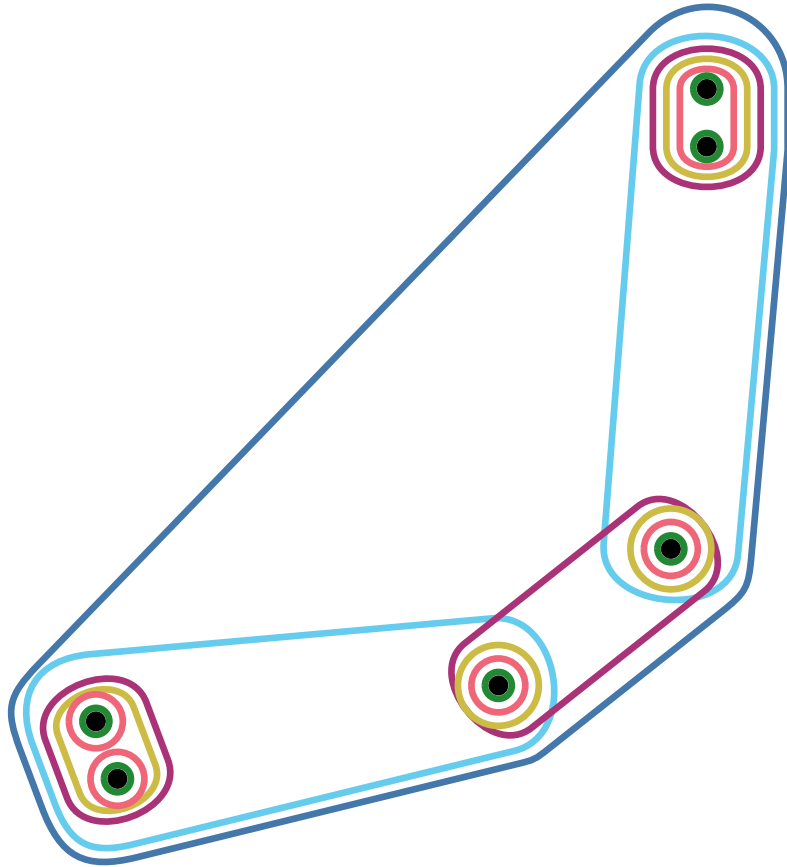
Hierarchical Clustering



Hierarchical Clustering



Hierarchical Clustering



Price of Hierarchy

$$\text{Ratio}_{k\text{-median}}(\mathcal{H}) := \max_{k=1,\dots,n} \frac{k\text{-median}(\mathcal{H}_k)}{k\text{-median}(\text{Opt}_k)}$$

$$\text{Apx}_{\text{Hierarchial } k\text{-median}} := \sup_{\text{points } I} \min_{\text{hierarchical clustering } \mathcal{H} \text{ on } I} \text{Ratio}_{k\text{-median}}(\mathcal{H})$$

Prior Work by Dai (2014):

$$1.0 \leq \text{Apx}_{\text{Hierarchial } k\text{-median}} \leq 16.0$$

Example starting code

```
@funsearch.evolve
```

```
def get_weighted_points() → list[tuple[float, np.ndarray]]:  
    """Return a new weighted clustering-problem, specified by a  
    list of weighted points. The returned tuple consists of the  
    weight of the point, and the point itself."""  
    weighted_points = [  
        (1.0, np.array([0, 0, 0, 0])),  
        (1e8, np.array([1, 0, 0, 0]))  
    ]  
    return weighted_points
```

Score: 1.0.

What FunSearch found

```
return [  
    (1.0, np.zeros(14)),  
    (1e10, np.ones(14)),  
    *[(1.0, np.eye(14)[i]) for i in range(7)],  
    *[(1.0, np.eye(14)[i]*-1) for i in range(7, 13)],  
    *[(1e10-i*1e9, np.linspace(i*0.1, (i+1)*0.1, 14, endpoint=False)) for i in  
range(7)],  
    (1e11, np.array([13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0])),  
    (1e12, np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13])),  
    (1e13, np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14])*10),  
    (1e14, np.array([14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1])*100),  
    (1e15, np.array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1])*1000),  
]
```

Score: 1.538

Trimming irrelevant lines

```
return [  
    (1.0, np.zeros(14)),  
    *[(1.0, np.eye(14)[i]) for i in range(7)],  
    *[(1.0, np.eye(14)[i] * -1) for i in range(7, 13)],  
    (1e10, np.linspace(0, 0.1, 14, endpoint=False)),  
]
```

Score: 1.538

Playing around with the instance

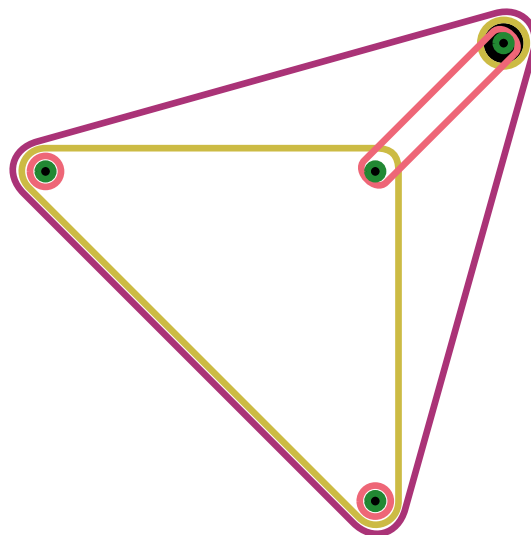
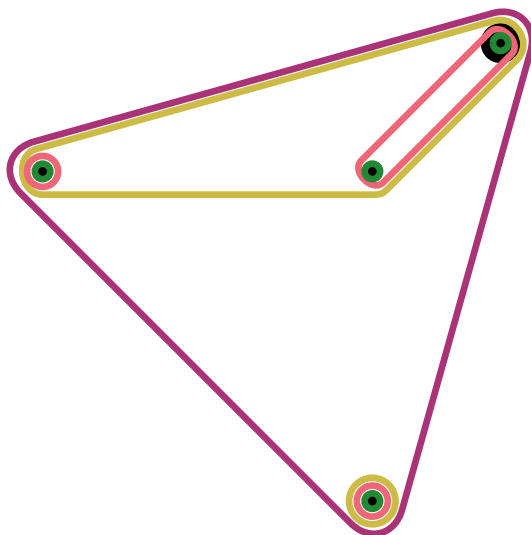
```
return [  
    (1.0, np.zeros(14)),  
    *[(1.0, -np.eye(14)[i]) for i in range(14)],  
    (1e10, np.ones(14) / 20),  
]
```

Score: 1.566

We could now prove things

Fix the dimension $d \geq 2$. Define $c := 2/(\sqrt{4d^2 + (3-d)^2} + d - 3)$. Let e_i be the i th d -dimensional standard-basis-vector. Consider the following weighted-discrete-kmedian-clustering-problem on $d + 2$ points:

$$(0, \dots, 0), \quad (c, \dots, c), \quad -e_1, \dots, -e_d,$$



Theorem:

$$\text{Apx}_{\text{Hierarchial } k\text{-median}} \geq \varphi = \frac{1 + \sqrt{5}}{2} \approx 1.618.$$

Results Overview

	Best-Fit	k -median	Knapsack	Gasoline
Previous Lower Bound	1.3	1.0	2.0	2.0
Local Search	1.478	1.36	1.93	2.11
FunSearch	1.497	1.538	646.93	3.05
FunSearch + Human	1.5	1.618	1.415^n	4.65
Known Upper Bound	1.7	16.0	2.0^n	None

Future Research

-

Future Research

- Automate pipeline fully using AITP?
-

Future Research

- Automate pipeline fully using AITP?
- Use FunSearch for developing *new* heuristics
-

Future Research

- Automate pipeline fully using AITP?
- Use FunSearch for developing *new* heuristics
- Low hanging fruit!

- Albers, S., Khan, A., & Ladewig, L. (2021). Best Fit Bin Packing with Random Order Revisited. *Algorithmica*, 83, 1–26. <https://doi.org/10.1007/s00453-021-00844-5>
- Dai, W. (2014). A 16-competitive algorithm for hierarchical median problem. *Science China Information Sciences*, 57(3), 1–7. <https://doi.org/10.1007/s11432-014-5065-0>
- Dósa, G., & Sgall, J. (2014). Optimal Analysis of Best Fit Bin Packing. In J. Esparza, P. Fraigniaud, T. Husfeldt, & E. Koutsoupias (Eds.), *Automata, Languages, and Programming: Automata, Languages, and Programming*.
- Romera-Paredes, B., Barekatin, M., Novikov, A., Balog, M., Kumar, M. P., Dupont, E., Ruiz, F. J., Ellenberg, J. S., Wang, P., Fawzi, O., & others. (2024). Mathematical discoveries from program search with large language models. *Nature*, 625(7995), 468–475.